

Product Requirements Document (PRD)

High-Contention Flash-Reservation & Adaptive Seat Inventory Engine

PLATFORM NAME	INVENTORY SIZING	TRAFFIC REGULATION	CORE GUARANTEE
TicketWala	Dynamic N Seats (Configurable)	Adaptive Request Balancing	0 Double-Bookings • Strict FCFS

Core Mission: Dynamic Capacity Scaling & Adaptive Request Balancing

TicketWala is engineered not for fixed toy constants, but as a production-scale distributed reservation engine. While 5,000+ requests against 200 units serves as our reference high-contention validation scenario, the engine dynamically supports arbitrary inventory capacities (50, 200, 1,000, 10,000+ units) and **automatically balances ingress requests against remaining available seats** using an adaptive Token Bucket shaper, circuit-breaking sold-out traffic, and guaranteeing strict First-Come-First-Served (FCFS) sub-second locks.

1. Executive Summary & Dynamic Capacity Architecture

In high-demand digital ticketing releases (concerts, sports, festivals, transit), demand typically exceeds inventory by multiple orders of magnitude. A fixed-rate system either starves high-capacity events or gets crushed by small-capacity micro-bursts.

The TicketWala Adaptive Architecture:

- Dynamic Capacity Parameterization:** Each event/show defines its discrete seat capacity N . The system initializes an atomic FIFO queue of exactly N seats without hardcoding.
- Automatic Request Balancing per Available Seats:** Ingress traffic is dynamically shaped based on current inventory scarcity: $\text{AdmissionRate}(t) = f(S_{\text{available}}(t), S_{\text{capacity}})$ When available inventory is high ($>50\%$), the ingress token bucket expands to admit high-volume parallel holding locks. As inventory nears depletion ($<10\%$), the broker applies progressive backpressure, serializing admissions into strict FIFO processing.
- Zero-Overhead Short-Circuiting:** Once all seats are held or confirmed ($S_{\text{available}} = 0$), incoming requests are rejected immediately at the in-memory boundary with clean `409 SOLD_OUT` responses in $< 2\text{ms}$, completely insulating database and worker layers from wasted compute.
- Instant Elastic Re-Opening:** If held seats expire or customers abandon checkout, seats are instantly returned to the FIFO queue head, and the adaptive governor immediately admits waiting requests.

2. Problem Statement Alignment & Feature Matrix

Challenge Requirement	TicketWala Engineering Mechanism	Dynamic Capability
Decouple write traffic from database	Pure Redis memory hot path using Lua scripts. PostgreSQL persistence occurs asynchronously via Redis Streams.	Decouples regardless of whether capacity is 50 or 50,000 seats.
Adaptive Request Balancing per Seats	Adaptive Token Bucket (Lua): Dynamically scales admission tokens and backpressure based on remaining seat count.	Prevents connection exhaustion; matches processing rate to available inventory.
Temporary holds via TTL counters	Server-side 120s TTL counter with monotonic version fencing. Client clocks ignored.	Configurable TTL per event tier (e.g., 60s flash vs 180s VIP).
Strict First-Come-First-Served (FCFS)	Redis FIFO List (<code>ticketwala:event:{id}:available_queue</code>). Popped in exact sequence via <code>RPOP</code> .	Deterministic arrival-order allocation down to microsecond execution.
Instant Release upon Abandonment/Timeout	<code>POST /release</code> or TTL expiration pushes unit back to queue head via <code>LPUSH</code> within < 5ms.	Immediately available for the next awaiting customer in line.
Guaranteed Eventual Consistency	Redis Streams consumer group worker commits idempotent SQL transactions to PostgreSQL 18.	Reconciliation engine verifies 100% state convergence.
Headless Load-Testing Suite	k6 test suite executing 5,000+ requests against parameterized seat counts.	Validates 0 double-bookings and sub-second holding locks under any contention scale.

3. System Goals, Non-Goals & Scope Guardrails

P0 Mandatory Core (Aligned with 18-Hour Mission)

- **P0 Configurable Seat Capacity:** API supports seeding arbitrary inventory pools $\setminus(N\setminus)$ dynamically.
- **P0 Adaptive Request Balancer:** Token Bucket shaper automatically tuning admission rate to seat availability.
- **P0 Strict FCFS Queue:** FIFO list allocation ensuring exact arrival-order assignment.
- **P0 Sub-Second Holding Locks:** 120s TTL holds with cryptographic token authentication.
- **P0 Instant Abandonment/Expiry Release:** Immediate FIFO re-queueing of cancelled seats.
- **P0 Asynchronous Relational Ledger:** Redis Streams → PostgreSQL 18 with idempotent replay.
- **P0 Independent Invariant Auditor:** Mathematical proof of zero double-allocations.

P1 Observability & Explicit Non-Goals

- **P1 Contention Observatory:** Dynamic visual grid displaying seat states (Green, Amber, Purple, Red).
- **P1 Live Worker Chaos Toggle:** Worker disconnect/reconnect resilience demonstration.
- **OUT NO Real Bank Merchant APIs:** Payment simulated via deterministic mock endpoint.
- **OUT NO Heavyweight Kafka/Kubernetes:** Redis Streams provides built-in consumer groups and pending lists without operational bloat.
- **OUT NO Multi-Region Sharding:** Single-master Redis guarantees atomic multi-key Lua execution.

4. Non-Negotiable System Invariants

Invariant 1: Absolute Capacity & Single Ownership Bound

For any configured inventory capacity $\setminus(N\setminus)$:

```
COUNT(Unit.status == 'HELD') + COUNT(Unit.status == 'CONFIRMED') + COUNT(Unit.status == 'AVAILABLE') == N  
All units u:  $|\{r \text{ in Reservations} : r.\text{unit\_id} == u.\text{id} \text{ and } r.\text{status} \text{ in } \{'HELD', 'CONFIRMED'\}\}| \leq 1$ 
```

Guarantee: Zero double-bookings under any concurrency level.

Invariant 2: Adaptive Ingress Governance & Strict FCFS Ordering

The system regulates incoming traffic dynamically. Requests that pass the adaptive admission gate are processed in exact arrival sequence. If capacity is exhausted, subsequent requests receive instant $\setminus(O(1)\setminus)$ rejections without degrading system performance.

Invariant 3: Instant Abandonment & Version Fence Safety

Abandoned holds are released immediately. If a delayed expiry job executes after a unit has been claimed by a newer reservation, monotonic version checking $\setminus(V_{\text{job}} < V_{\text{current}}\setminus)$ ensures the job executes as a safe no-op.

Invariant 4: Cryptographic Token Proof & Immutability

Confirmation and release require the 128-bit secret token matching the SHA-256 hash stored at hold creation time. Once confirmed, status is terminal and cannot be revoked by expiry.

Invariant 5: Guaranteed Eventual Consistency

All in-memory state transitions append an immutable event to Redis Streams. The PostgreSQL worker commits events idempotently. Relational storage converges with in-memory state within < 500ms.

5. Hackathon Judge-Readiness Scorecard (100 Points)

Evaluation Category	Points	Official Challenge Requirement Tested	TicketWala Implementation Proof
Innovation	25	Decoupled write tier, in-memory broker, Adaptive Request Balancing per seats, Monotonic Version Fencing.	Redis Lua atomic engine + Adaptive Token Bucket shaper + Monotonic Version Fencing + Live Invariant Auditor.
Technical Execution	25	Sub-second locks, zero double-bookings, strict FCFS, headless load testing (5,000+ req).	Automated k6 load test sustaining 5,000+ concurrent requests with raw CSV/JSON telemetry and 0 oversell.
Real-World Utility	25	Dynamic capacity sizing, instant release on timeout/abandonment, crash resilience, eventual consistency.	Configurable seat capacity, resilient worker with `XAUTOCLAIM` recovery, live worker kill demonstration, and automated reconciliation.
UI/UX & Presentation	25	Intuitive customer flow, professional operations dashboard, compelling live demo narrative.	Next.js 15 customer booking flow + real-time Contention Observatory with live seat matrix and chaos controls.